Daily Quant Digest: 2026-10-31

Foundations Restart, day 6. Yesterday we shared out identical things with stars and bars, met the forward contract, and answered range questions with prefix sums. Today: what happens to probabilities when you learn something (conditional probability), where the forward price actually comes from (no-arbitrage and covered interest parity), and how merge sort works, used to count how scrambled a list is.

Today's topics

Track	Topic	Difficulty
Probability & Statistics	FR Lesson 6: conditional probability, shrinking the room	Easy-Medium
Derivatives	FR Lesson 6: pricing a forward by no-arbitrage: cost of carry, covered interest parity, forward points	Easy-Medium
Algorithms & DS	FR Lesson 6: sorting: merge sort, `std::sort`, custom comparators; counting inversions	Medium
Music	Deep Purple, "Smoke on the Water" (1972)	

---

1. Probability & Statistics: Shrinking the Room

> **Where we are.** Lesson 1: the sample space is a **room of possible worlds**, and when the worlds are equally likely, a probability is a share of the room: (worlds where the event happens) / (all worlds). Lesson 2: "A and B" is the overlap $A \cap B$. Lessons 3 to 5: tools for counting worlds. Today we ask: **what happens to these shares when somebody tells you something?**

1.1 The central picture: news moves the walls

Picture the room of possible worlds again. Now a friend walks in and gives you a piece of news: "the result is even", or "the trader hit her stop-loss". Every world that **disagrees** with the news is now impossible. So walls slide in, and you are standing in a **smaller room** that holds only the worlds that agree with the news.

Two things do **not** change:

- The worlds inside the small room are the same worlds as before.
- If they were equally likely before, they are still equally likely to each other now.

So the new probability of an event is simply its **share of the small room**. That is the whole idea of conditional probability. Everything else today is bookkeeping.

In the picture	In the maths
The big room	The sample space Ω
The news	The event B we condition on
The small room after the walls move	B itself, the new "everything"
The part of A still inside the small room	$A \cap B$
The share of the small room taken by A	$P(A \mid B)$, read "probability of A given B"

Where the picture stops working: it assumes the news is simply **true** and that you learned it in a fixed, known way. If **how** you heard the news depends on the world you are in, the walls move differently. The interview question below is exactly about that.

1.2 Count first: 1,000 traders

Imagine a firm with **1,000 traders**. 400 trade FX and 600 trade rates. Last month, 100 of the FX traders hit their stop-loss, and 60 of the rates traders did.

	Hit stop-loss	No stop-loss	Total
FX	100	300	400
Rates	60	540	600
Total	160	840	1,000

Pick a trader at random.

- **Before any news.** $P(\text{stop}) = 160/1{,}000 = 16\%$.
- **News: "she trades FX".** The walls move in to the FX row: 400 traders. Of those, 100 hit a stop. $P(\text{stop} \mid \text{FX}) = 100/400 = 25\%$.
- **Different news: "she hit her stop-loss".** Now the walls move in to the **stop-loss column**: 160 traders. Of those, 100 trade FX. $P(\text{FX} \mid \text{stop}) = 100/160 = 62.5\%$.

Look at the last two lines. Same overlap (the 100 FX traders who were stopped out), **different small rooms** (400 vs 160), so different answers: 25% and 62.5%. $P(A \mid B)$ and $P(B \mid A)$ are different questions. Mixing them up is the single most common mistake in this topic.

1.3 The formula, one step at a time

In words: the chance of A given B is the part of A that lives inside B, as a share of B.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)}, \qquad \text{as long as } P(B) > 0.$$

Symbols. A = the event we care about. B = the news. $A \cap B$ = worlds where both happen. The vertical bar $\mid$ is read "given".

Read aloud: "the probability of A given B is the probability of A and B, divided by the probability of B."

With numbers. A = stop-loss, B = FX. $P(A \cap B) = 100/1{,}000 = 0.10$, $P(B) = 400/1{,}000 = 0.40$, so $P(A \mid B) = 0.10/0.40 = 0.25$. The 1,000 cancels: dividing by $P(B)$ is just "rescale so the small room counts as 100%".

Why divide? Inside the small room, the shares must add up to 1 again. $P(B)$ is the size of the small room measured in the old units, so dividing by it resets the units.

Turned around: the multiplication rule. Multiply both sides by $P(B)$:

$$P(A \cap B) = P(B) \times P(A \mid B).$$

In words: to get the chance that both happen, take the chance of the first, then the chance of the second **once the walls have moved**.

With numbers: two aces in a row. Deal 2 cards from a 52-card deck. $P(\text{first is an ace}) = 4/52$. Given that, 51 cards remain and 3 are aces, so $P(\text{second ace} \mid \text{first ace}) = 3/51$. Together:

$$\frac{4}{52} \times \frac{3}{51} = \frac{12}{2{,}652} = \frac{1}{221}.$$

Check with Lesson 4: pairs of aces $\binom{4}{2} = 6$ out of $\binom{52}{2} = 1{,}326$ hands, and $6/1{,}326 = 1/221$. Two methods, same answer.

1.4 A small dice example

Roll two dice (36 equally likely worlds, Lesson 1). Without news, $P(\text{at least one six}) = 11/36 \approx 0.31$.

News: the total is 8. The small room is $\{(2,6), (3,5), (4,4), (5,3), (6,2)\}$: 5 worlds. Two of them contain a six. So

$$P(\text{at least one six} \mid \text{sum } 8) = \frac{2}{5} = 0.40.$$

The news raised the chance from 31% to 40%. Note what we did **not** do: we did not keep 36 on the bottom. The denominator is always the size of the **new** room.

1.5 Common mistakes

- **Flipping the condition.** "Most stopped-out traders trade FX" (62.5%) does not mean "most FX traders get stopped out" (25%). In a courtroom this mistake has a name, the prosecutor's fallacy. Lesson 9 (Bayes' rule) is the tool for flipping correctly.
- **Forgetting to shrink the denominator.** Dividing the overlap by the whole room gives $P(A \cap B)$, not $P(A \mid B)$.
- **Ignoring how the news was produced.** "At least one card is an ace" and "this particular card is the ace of spades" sound similar but are different news, with different small rooms. See below.

1.6 Self-check

1. Roll one die. Given the result is even, what is the probability it is a 6?
2. From the 1,000-trader table: given a trader did **not** hit her stop-loss, what is the probability she trades rates?
3. Deal 2 cards. What is the probability both are hearts?

1.7 Interview question: two cards, two kinds of news

(A classic opener at SIG, Optiver and Jane Street. It tests whether you define the small room carefully instead of trusting your gut.)

Two cards are dealt from a well-shuffled 52-card deck.

a. You are told: "at least one of your two cards is an ace." What is the probability that both are aces?

b. Instead you are told: "one of your cards is the ace of spades." What is the probability that both are aces?

c. Instead you are told: "the first card dealt is an ace." What is the probability that both are aces?

d. Explain in one or two sentences why (a) and (b) differ.

1.8 Solution

The room. All 2-card hands: $\binom{52}{2} = 1{,}326$, equally likely.

(a) At least one ace. Small room: hands with at least one ace. Count the complement (Lesson 2): hands with **no** ace are $\binom{48}{2} = 1{,}128$, so the small room has $1{,}326 - 1{,}128 = 198$ hands. Hands with two aces: $\binom{4}{2} = 6$.

$$P(\text{both aces} \mid \text{at least one ace}) = \frac{6}{198} = \frac{1}{33} \approx 0.030.$$

(b) Holding the ace of spades. Small room: hands that contain the ace of spades. The other card is any of the 51 others, so 51 hands. Of those, 3 have another ace.

$$P(\text{both aces} \mid \text{ace of spades}) = \frac{3}{51} = \frac{1}{17} \approx 0.059.$$

(c) First card is an ace. Use ordered deals: $52 \times 51 = 2{,}652$. Small room: first card is an ace, $4 \times 51 = 204$ deals. Both aces: $4 \times 3 = 12$. So $12/204 = 1/17$. (Or simply: given the first is an ace, 3 aces remain among 51 cards.)

(d) Why (a) is smaller. "At least one ace" is **weak** news. Its small room has 198 hands, and 192 of them (4×48) have exactly one ace. The two-ace hands are only 6 of the 198. "The ace of spades" is **specific** news: it pins down one card and leaves the other completely free, so the question becomes "is a random card from the other 51 an ace?" Naming the ace throws out all the single-ace hands built on the other three aces, which makes the two-ace hands a bigger share.

Interview trick. Whenever the news is "a specific named thing happened", the answer is usually "what is left over is just a random draw from what remains". Whenever the news is "at least one", count the complement for the small room.

Sanity check. (a): $192 + 6 = 198$, so the room is fully accounted for. Both answers are bigger than the unconditional $1/221$, as they must be: any news about an ace makes two aces more likely. And (b) and (c) agree, because in both cases one ace is fixed and one card is still free. I also checked all 1,326 hands by computer: 198, 6, 51 and 3.

Self-check answers. (1) Small room $\{2, 4, 6\}$, so $1/3$. (2) Small room: 840 traders, 540 in rates, $540/840 = 9/14 \approx 0.643$. (3) $\frac{13}{52} \times \frac{12}{51} = \frac{156}{2{,}652} = \frac{1}{17}$.

---

2. Derivatives: Where the Forward Price Comes From

> **Where we are.** Lesson 1: money grows with interest, $1 + rt$ for simple interest over a fraction t of a year. Lesson 2: money later is worth less than money now. Lesson 4: EURUSD 1.1650 means 1 euro costs 1.1650 dollars, and a pip is 0.0001. Lesson 5: a forward fixes today the rate for an exchange on a later date. Yesterday a Hong Kong importer bought EUR 5 million for 30 January at **1.1700** while spot was **1.1650**. Today: why 1.1700, and not 1.1650 or 1.2000?

2.1 The scene

The finance director looks at the deal again and asks the bank's salesperson: "Your rate is 50 pips above spot. Does your bank think the euro will go up?"

The honest answer is no. The forward trader at the bank does not forecast anything. As soon as she sells the euros forward, she has a risk: if the euro rises, she loses. So she **builds the same thing by herself**, today, with three plain trades that anyone with a bank account understands. The forward price is simply what those three trades cost. It is arithmetic, not opinion.

2.2 The central picture: two roads to the same village

Think of two roads from today to the same village: "**having EUR 5 million on 30 January, with no risk**".

- **Road 1:** sign a forward at price F. Pay $F \times 5$ million dollars on 30 January.
- **Road 2:** do it yourself. Borrow dollars today, change them into euros at spot, and put the euros in the bank until January.

If both roads arrive at exactly the same place with no risk, they **must cost the same**. If one were cheaper, every bank would travel the cheap road and sell the expensive one, until the gap closed. That is the whole of "no-arbitrage pricing".

Road 2 has a twist. While the euros sit in the bank, they **earn euro interest**. Meanwhile the dollars you borrowed **cost dollar interest**. Holding something until a later date has a cost (or a benefit): this is the **cost of carry**. For gold, the carry is the interest on the money plus the vault fee. For a currency, the "vault" is a deposit account that **pays** you.

In the picture	In the maths
The village	EUR 5m delivered on the forward date
Road 1, the toll you pay	Forward price F
Road 2: borrow USD, buy EUR, deposit	Spot S, USD rate r_{USD}, EUR rate r_{EUR}
The cost of the journey on road 2	Cost of carry: USD interest paid minus EUR interest earned
Roads must cost the same	Covered interest parity

2.3 Building the forward by hand

Numbers (simple, realistic): spot $S = 1.1650$, 92 days from 30 October to 30 January, USD rate 3.60%, EUR rate 2.00%. Money markets in both currencies count days as actual/360, so $t = 92/360 = 0.25556$. (Real deals settle spot two business days later; we ignore that today.)

Step 1: how many euros to deposit today? We want exactly EUR 5,000,000 in January. A deposit grows by $1 + 0.02 \times 0.25556 = 1.005111$. So deposit

$$\frac{5{,}000{,}000}{1.005111} = \text{EUR } 4{,}974{,}574.$$

Step 2: buy those euros at spot. $4{,}974{,}574 \times 1.1650 = \text{USD } 5{,}795{,}379$.

Step 3: borrow those dollars for 92 days. We owe $5{,}795{,}379 \times (1 + 0.036 \times 0.25556) = 5{,}795{,}379 \times 1.0092 = \text{USD } 5{,}848{,}697$ in January.

Result. On 30 January we receive EUR 5,000,000 and pay USD 5,848,697. That is a rate of

$$\frac{5{,}848{,}697}{5{,}000{,}000} = 1.16974.$$

So the **fair forward is about 1.1697**. The bank quoted 1.1700, about 2.6 pips more. On EUR 5 million that is about USD 1,300: the bank's margin for doing the work.

2.4 The formula

In words: start at spot, grow it by the quote currency's interest, and shrink it by the base currency's interest.

$$F = S \times \frac{1 + r_{\text{quote}}\, t}{1 + r_{\text{base}}\, t}.$$

Symbols. S = spot (quote currency per 1 unit of base currency). F = forward rate. r_{quote} = interest rate of the quote currency (USD in EURUSD). r_{base} = interest rate of the base currency (EUR). t = time in years.

Read aloud: "forward equals spot, times one plus the dollar rate times time, over one plus the euro rate times time."

With numbers. $1.1650 \times \frac{1.0092}{1.005111} = 1.1650 \times 1.004068 = 1.16974$. Same as Section 2.3, as it must be: the formula **is** the three steps.

Why "covered" interest parity? Because the exchange-rate risk of the do-it-yourself road is **covered** by locking in both ends. With continuous compounding (Lesson 1) it reads $F = S\, e^{(r_{\text{quote}} - r_{\text{base}})T}$.

2.5 Forward points: the way dealers quote it

Dealers do not quote the full forward. They quote the **difference** in pips:

$$\text{forward points} = (F - S) \times 10{,}000 = (1.16974 - 1.1650) \times 10{,}000 \approx +47.4.$$

A quick rule of thumb: points $\approx S \times (r_{\text{quote}} - r_{\text{base}}) \times t \times 10{,}000 = 1.1650 \times 0.016 \times 0.25556 \times 10{,}000 \approx 47.6$. Close, because the rates are small.

The rule that confuses everyone. The currency with the **lower** interest rate trades at a forward **premium** (it costs more forward), and the higher-rate currency at a **discount**. EUR has the lower rate, so EURUSD forward is above spot. Common sense: if you hold euros you earn less interest, so the forward must pay you back for that, or everybody would hold dollars and lock in the way back for free.

USDJPY. Spot 152.00, USD 3.60%, JPY 0.75% (same day count for simplicity). Here USD is the base and has the **higher** rate:

$$F = 152.00 \times \frac{1 + 0.0075 \times 0.25556}{1 + 0.036 \times 0.25556} \approx 150.90.$$

Points are about -110 (JPY pips are 0.01). The forward is **below** spot. This is not a forecast that the yen will rise; it is just interest.

2.6 What if someone quotes the wrong price?

Suppose a careless bank shows a 3-month EURUSD forward at **1.1750** (fair is 1.1697). Forward too high means: **sell** at the expensive forward, **build** the cheap one.

1. Sell EUR 5m forward to that bank at 1.1750: you will receive USD 5,875,000 in January.
2. Do Section 2.3: borrow USD 5,795,379, buy EUR 4,974,574, deposit. In January you have EUR 5m to deliver and owe USD 5,848,697.
3. January: deliver the euros, receive USD 5,875,000, repay USD 5,848,697. **Profit about USD 26,300**, with no view on EURUSD at all.

If the forward were too **low** (say equal to spot, 1.1650), you would do the reverse (borrow euros, sell them, deposit dollars, buy euros forward) and make about USD 23,700.

2.7 Where the theory bends

For decades covered interest parity held almost exactly. Since the 2008 crisis there is a small, persistent gap, called the **cross-currency basis**, especially for EUR and JPY against USD. The reason is that the "free" arbitrage in Section 2.6 uses a bank's balance sheet, and after 2008 balance sheet became expensive under new regulation. The gap tends to widen at quarter-ends and in dollar funding stress, such as late 2008 and March 2020, when central banks used dollar swap lines to calm it. For interviews: CIP holds to a few pips in normal times, and deviations are a measure of the cost of dollar funding.

2.8 Likely interview questions

Q1. EURUSD spot is 1.1700, the USD rate is 4% and the EUR rate is 2%, simple interest. What is the 1-year forward? $1.17 \times 1.04/1.02 \approx 1.1929$, about +229 pips. Lower-rate EUR is at a premium.

Q2. The 3-month USDJPY forward is below spot. Does the market expect the yen to strengthen? No. The forward is set by the interest difference (USD rate above JPY rate), not by expectations. In fact, historically high-rate currencies have often **not** fallen as much as the forward implies, which is the basis of the carry trade.

Q3. The forward is quoted above fair value. Walk me through the arbitrage. Sell the forward. Borrow the quote currency, buy the base currency at spot, deposit it. At maturity deliver the base currency into the forward, use the proceeds to repay the loan, keep the difference.

---

3. Algorithms & Data Structures: Merge Sort and Counting Inversions

> **Where we are.** Lesson 1: Big-O, and why $n \log n$ beats n^2. Lesson 2: `std::vector` and references. Lesson 3: scanning and updating arrays in place. Lesson 4: hash maps. Lesson 5: prefix sums. Today: sorting, the most used algorithm in programming, and one problem where knowing **how** the sort works gives a fast answer.

3.1 The central picture: two clerks and a pile of tickets

A clerk has a pile of 8 trade tickets to sort by ticket number. It is slow to do alone, so she splits the pile in two and gives each half to a helper. Each helper does the same: split, hand off. Eventually someone holds a single ticket, which is already "sorted".

Now the work comes back up. Two sorted piles, face up. To **merge** them, look at the two top tickets, take the **smaller** one, put it on the output pile. Repeat. Each step moves one ticket, so merging two piles with m tickets in total takes m steps.

In the picture	In the code
Splitting the pile in half	`mid = lo + (hi - lo) / 2`
A helper sorting a half	The recursive call
One ticket alone	Base case: size 0 or 1 is sorted
Comparing two top tickets	`a[i] <= a[j]`
The output pile	The scratch buffer `buf`

Why $n \log n$? Draw the splitting as levels. 8 tickets: level 1 has 2 piles of 4, level 2 has 4 piles of 2, level 3 has 8 piles of 1. That is $\log_2 8 = 3$ levels. At **every** level, all n tickets are merged once, so each level costs about n steps. Total: $n \times \log_2 n$. For $n = 1{,}000{,}000$, that is about 20×10^6 steps, against about 5×10^{11} for comparing every pair.

Merge sort needs an extra buffer of size n, so it uses $O(n)$ space. It is also **stable**: equal values keep their original order, because on a tie we take from the left pile first.

3.2 The problem

A client sends trades numbered in the order she sent them. They pass through several network paths and arrive at our gateway scrambled. To measure **how scrambled** the arrival is, count the **inversions**: pairs of positions $i < j$ with $a[i] > a[j]$ (a later ticket number arrived before an earlier one).

- **Input:** an array `a` of n integers, $0 \le n \le 2 \times 10^5$. Values may repeat; equal values are **not** an inversion.
- **Output:** the number of inversions.

Examples.

Input	Inversions	The pairs
[3, 1, 4, 5, 2]	4	(3,1), (3,2), (4,2), (5,2)
[2, 4, 1, 3, 5]	3	(2,1), (4,1), (4,3)
[1, 2, 3, 4]	0	already in order
[5, 4, 3, 2, 1]	10	every pair: $\binom{5}{2}$

The worst case, a fully reversed array, has $\binom{n}{2} = n(n-1)/2$ inversions. In statistics this count is the **Kendall tau distance** between two rankings, a common way to compare how two signals rank the same stocks.

3.3 Brute force first

Check every pair: two nested loops, $\binom{n}{2}$ comparisons, $O(n^2)$ time, $O(1)$ space. For $n = 2 \times 10^5$ that is about 2×10^{10} comparisons: far too slow. We need to count pairs **without looking at them one by one**.

3.4 The insight: inversions show up when a ticket jumps the queue

Split the array into a left half and a right half. Every inversion is one of three kinds:

1. **Both in the left half.** Counted by the recursive call on the left.
2. **Both in the right half.** Counted by the recursive call on the right.
3. **One in each half** (a "cross" pair). These are new.

Here is the key. When we merge, both halves are **already sorted**. Suppose the left top is $a[i]$ and the right top is $a[j]$, and $a[j] < a[i]$. We take $a[j]$. But the left pile is sorted, so **every** ticket still waiting on the left (from i up to `mid - 1`) is at least $a[i]$, so all of them are bigger than $a[j]$. And all of them sat **before** $a[j]$ in the original array. That is `mid - i` cross inversions, counted in **one** step.

Trace on [3, 1, 4, 5, 2].

- Left [3, 1]: split into [3] and [1]. Merge: 1 jumps ahead of 3, +1. Gives [1, 3].
- Right [4, 5, 2]: split into [4] and [5, 2]. [5, 2]: 2 jumps over 5, +1, gives [2, 5]. Merge [4] with [2, 5]: 2 jumps over 4, +1, gives [2, 4, 5]. Right total: 2.
- Final merge of [1, 3] with [2, 4, 5]: take 1. Now 3 vs 2: 2 jumps ahead, and one ticket (the 3) is still waiting on the left: +1. Then 3, 4, 5 go without jumps.
- Total: $1 + 2 + 1 = 4$. Matches the table.

Ties. On $a[i] = a[j]$ we take the **left** one first (`<=`). That way an equal value never "jumps" and is never counted, which is what the problem wants.

Complexity. We only add a counter to merge sort, so $O(n \log n)$ **time** and $O(n)$ **extra space**. The count can reach 2×10^{10}, more than an `int` holds (about 2.1×10^9), so it must be a `long long`.

3.5 Sorting in everyday C++: `std::sort` and comparators

In real code you rarely write a sort. `std::sort(v.begin(), v.end())` sorts in $O(n \log n)$. It is a hybrid of quicksort, heapsort and insertion sort, and it is **not stable**. If you need equal items to keep their order, use `std::stable_sort`, which is typically a merge sort like ours.

To sort by your own rule, pass a **comparator**: a function `comp(x, y)` that returns `true` when `x` must come **before** `y`. Two rules:

- It must behave like "less than": `comp(x, x)` must be false. **Never use** `<=` in a comparator; it breaks `std::sort` and can even crash it.
- For ties on the first key, compare a second key.

The code below sorts bids like an order book: **highest price first**, and at the same price, **earliest time first** (price-time priority).

3.6 C++ solution

```cpp
// Daily Quant Digest 2026-10-31: Sorting (merge sort, std::sort, custom comparators).
// Problem: count the pairs of trades that arrived out of order (inversions).
// Compile: g++ -std=c++17 -O2 -Wall 2026-10-31.cpp && ./a.out
#include <algorithm>
#include <cassert>
#include <iostream>
#include <random>
#include <string>
#include <vector>

// Brute force: look at every pair (i, j) with i < j. O(n^2) time, O(1) space.
long long countInversionsBrute(const std::vector<int>& a) {
    long long count = 0;
    const std::size_t n = a.size();
    for (std::size_t i = 0; i < n; ++i)
        for (std::size_t j = i + 1; j < n; ++j)
            if (a[i] > a[j]) ++count;
    return count;
}

// Sort a[lo, hi) with merge sort and return the number of inversions inside it.
// buf is scratch space of the same size as a, shared by every call.
long long sortAndCount(std::vector<int>& a, std::vector<int>& buf,
                       std::size_t lo, std::size_t hi) {
    if (hi - lo <= 1) return 0;                 // 0 or 1 element: sorted, no inversions
    const std::size_t mid = lo + (hi - lo) / 2;

    long long count = sortAndCount(a, buf, lo, mid)   // inversions inside the left half
                    + sortAndCount(a, buf, mid, hi);  // inversions inside the right half

    // Merge the two sorted halves, counting the "cross" inversions.
    std::size_t i = lo, j = mid, k = lo;
    while (i < mid && j < hi) {
        if (a[i] <= a[j]) {
            buf[k++] = a[i++];                  // left value is smaller: no inversion
```

```cpp
        } else {
            // a[j] jumps ahead of a[i] and of every value still waiting on the left.
            count += static_cast<long long>(mid - i);
            buf[k++] = a[j++];
        }
    }
    while (i < mid) buf[k++] = a[i++];          // copy whatever is left over
    while (j < hi)  buf[k++] = a[j++];
    std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);
    return count;
}

// Public entry point: takes the array by value, so the caller's copy is untouched.
long long countInversions(std::vector<int> a) {
    std::vector<int> buf(a.size());
    return sortAndCount(a, buf, 0, a.size());
}

// A small demo of std::sort with a custom comparator.
struct Trade {
    std::string id;
    double price;
    int time;   // seconds after the open
};

int main() {
    // 1) Worked examples from the text.
    struct Case { std::vector<int> a; long long expected; };
    std::vector<Case> cases = {
        {{3, 1, 4, 5, 2}, 4},
        {{2, 4, 1, 3, 5}, 3},
        {{1, 2, 3, 4}, 0},
        {{5, 4, 3, 2, 1}, 10},
        {{2, 2, 1}, 2},          // equal values are NOT an inversion
        {{}, 0},
    };
    for (const auto& c : cases) {
        long long got = countInversions(c.a);
        std::cout << "inversions = " << got << " (expected " << c.expected << ")\n";
        assert(got == c.expected);
        assert(countInversionsBrute(c.a) == c.expected);
    }

    // 2) Randomized cross-check against brute force, and check the sort itself.
    std::mt19937 rng(20261031);
    std::uniform_int_distribution<int> lenDist(0, 40), valDist(-5, 5);
    for (int t = 0; t < 20000; ++t) {
        std::vector<int> a(lenDist(rng));
        for (auto& x : a) x = valDist(rng);
        assert(countInversions(a) == countInversionsBrute(a));

        std::vector<int> mine = a, expected = a, buf(a.size());
        sortAndCount(mine, buf, 0, mine.size());
        std::sort(expected.begin(), expected.end());
        assert(mine == expected);
    }

    // 3) Large reversed input: n(n-1)/2 is about 2 * 10^10, too big for int.
    const int n = 200000;
    std::vector<int> rev(n);
    for (int i = 0; i < n; ++i) rev[i] = n - i;
    assert(countInversions(rev) == static_cast<long long>(n) * (n - 1) / 2);

    // 4) std::sort with a custom comparator: best price first (highest bid),
    //    and at the same price, earliest time first.
    std::vector<Trade> bids = {
```

```
        {"B1", 1.1650, 30}, {"B2", 1.1652, 45}, {"B3", 1.1650, 10}, {"B4", 1.1652, 20}};
    std::sort(bids.begin(), bids.end(), [](const Trade& x, const Trade& y) {
        if (x.price != y.price) return x.price > y.price;   // higher price first
        return x.time < y.time;                             // then earlier time
    });
    std::cout << "bid queue:";
    for (const auto& b : bids) std::cout << ' ' << b.id;
    std::cout << '\n';
    assert(bids[0].id == "B4" && bids[1].id == "B2" && bids[2].id == "B3" && bids[3].id == "B1");

    std::cout << "All tests passed.\n";
    return 0;
}
```

Output:

```
inversions = 4 (expected 4)
inversions = 3 (expected 3)
inversions = 0 (expected 0)
inversions = 10 (expected 10)
inversions = 2 (expected 2)
inversions = 0 (expected 0)
bid queue: B4 B2 B3 B1
All tests passed.
```

The randomized test compares the fast count with brute force on 20,000 small random arrays full of repeats, and also checks that our merge sort gives exactly what `std::sort` gives. The reversed test with $n = 200{,}000$ produces 19,999,900,000 inversions, which would overflow an `int`.

3.7 Walkthrough of the C++

Half-open ranges `[lo, hi)`. `sortAndCount` works on positions `lo` up to but **not including** `hi`. This is the same convention as `begin()` and `end()` in the STL. It makes the sizes easy (`hi - lo`) and the two halves `[lo, mid)` and `[mid, hi)` fit together with no overlap and no gap.

`lo + (hi - lo) / 2` instead of `(lo + hi) / 2`. Both give the middle, but `lo + hi` can overflow for huge indices. A good habit, and interviewers notice it.

`std::vector<int>& a, std::vector<int>& buf`. Non-const **references** (Lesson 2): the function sorts the caller's vector in place and writes into the shared buffer. No copies at each level. Allocating one buffer once in `countInversions` and passing it down is much faster than creating a new vector in every call.

`long long countInversions(std::vector<int> a)`. Here `a` is taken **by value**, a deliberate copy. Merge sort rearranges the array, and the caller probably does not want their data sorted behind their back. Compare with `countInversionsBrute(const std::vector<int>& a)`, which only reads, so it takes a const reference.

`buf[k++] = a[i++];`. The **post-increment** `i++` gives the old value of `i` to the expression, then adds 1. So this line means "copy `a[i]` into `buf[k]`, then move both pointers forward". Compact and very common in merge code.

`count += static_cast<long long>(mid - i);`. `mid - i` is a `std::size_t` (unsigned). The cast makes the intent clear and keeps all counting in signed 64-bit.

`std::copy(buf.begin() + lo, buf.begin() + hi, a.begin() + lo);`. Copies the merged range back. `buf.begin() + lo` is an **iterator** moved forward by `lo` positions; vector iterators support this arithmetic like pointers. `std::copy(first, last, dest)` copies `[first, last)` starting at `dest`.

`struct Trade { ... };` A plain record type. `{"B1", 1.1650, 30}` builds one with brace initialization, filling the fields in order.

The lambda `[](const Trade& x, const Trade& y) { ... }`. A lambda is a small function written inline. The `[]` is the **capture list** (empty: it uses nothing from outside). It takes two trades by const reference and returns `true` when `x` should come first. `std::sort` calls it every time it needs to compare two elements. Note `>` on price: that flips the usual order to put the highest bid first.

`assert(mine == expected);` Two vectors compare equal with `==` when they have the same size and the same elements in the same order.

Where else this idea appears. "Count pairs with some order property" problems (pairs where $a[i] > 2\,a[j]$, smaller numbers after each element) use the same divide-and-merge counting. Later we will solve them again with a Fenwick tree (Lesson 35).

---

4. Music: Deep Purple, "Smoke on the Water" (1972)

Context

"Smoke on the Water" is on Deep Purple's album *Machine Head*, released in March 1972. It is credited to all five members of the classic line-up: **Ritchie Blackmore** (guitar), **Ian Gillan** (vocals), **Roger Glover** (bass), **Jon Lord** (organ) and **Ian Paice** (drums). It was not planned as a single, but it was released as one in 1973 and reached number 4 on the US Billboard Hot 100.

The story behind it

In December 1971 the band went to Montreux, on Lake Geneva in Switzerland, to record the album with the Rolling Stones' mobile recording studio, a studio built into a truck. They planned to use the Montreux Casino, which was closing for the winter after one last concert by **Frank Zappa and the Mothers of Invention**.

During that concert, on 4 December, someone in the crowd fired a flare gun into the ceiling. The casino caught fire and burned down. The band watched the smoke spread across the lake. Claude Nobs, who ran the Montreux Jazz Festival, helped get people out, and he is mentioned by name in the song.

With their venue gone, they recorded where they could, first in a local theatre (until neighbours complained about the noise), then in the empty corridors of the closed Grand Hotel, with mattresses against the walls to deaden the sound. The song simply tells this story, almost like a news report. Roger Glover has said the title came to him when he woke up thinking of the smoke over the water.

Why people love it

The riff. It is four short phrases, easy enough that it is one of the first things many people ever play on guitar, and yet it sounds huge. It is heavy rock at its simplest: no showing off, just a groove that makes you nod your head.

What to listen for

- **The riff's sound.** Blackmore plays two notes at a time (a "fourth" apart), plucking with his fingers instead of a pick. That is why it sounds thick and slightly hollow, not like a single-note line.
- **The build-up.** The instruments come in one at a time over the riff: the guitar alone, then the drums starting quietly on the hi-hat, then the bass and organ, and finally the full band. Notice how the tension grows before the singing starts.
- **Jon Lord's organ.** A Hammond organ played loud and distorted through a guitar amplifier. It often sounds like a second guitar.
- **The guitar solo.** Blackmore's solo in the middle is melodic and controlled, built from short phrases rather than speed.
- **The rhythm section.** Ian Paice keeps it steady but swinging, never rushing. The song sits at a slow, heavy tempo and owes much of its weight to that patience.